Operação do circuito de debounce VHDL

1. Introdução:

A utilização de chaves mecânicas em circuitos digitais apresenta o problema do repique, ou em inglês “bouncing”, exigindo assim providências para se estabilizar a entrada por meio de circuitos anti-repique ou, em inglês, “debouncing”.

Existem várias soluções para esse problema, tanto analógicas como digitais. Aqui analisaremos uma solução digital. A ideia é só passar o sinal adiante se ele ficar inalterado durante um certo número de períodos do relógio. Isso involve um comparador para comparar amostras consecutivas do sinal oriundo do dispositivo mecânico e um contador para validar uma certa quantidade de amostras inalteradas.

Existem inúmeras propostas de solução para esse bem conhecido problema. A que será apresentada aqui foi baseada no artigo apresentado em:

https://www.digikey.com/eewiki/pages/viewpage.action?pageId=4980758

2. O Circuito

A Figura 1 abaixo ilustra o circuito típico utilizado.

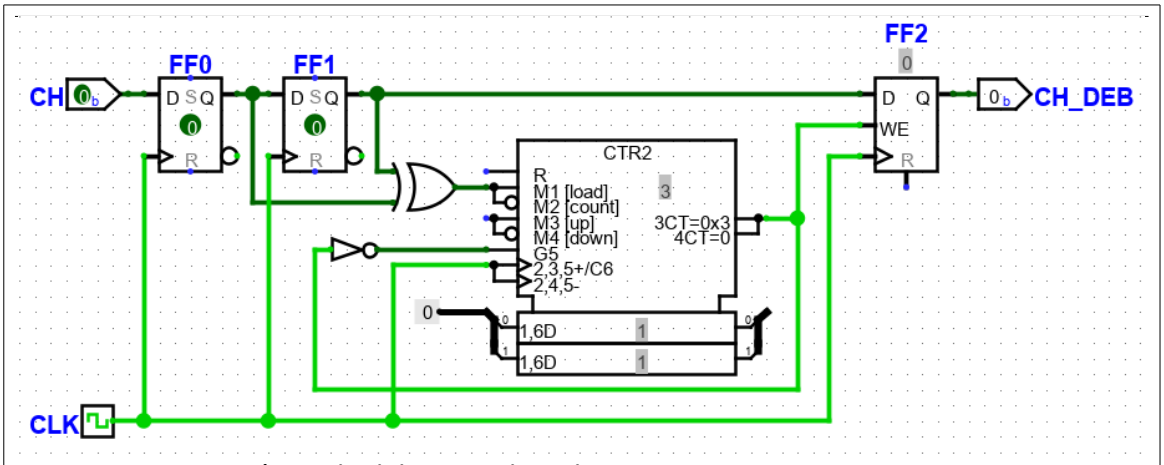

Figura 1: Circuito típico de debounce digital.

Vamos supor que a chave CH estivesse a muito tempo no estado zero e então é comutada para um. A análise será igualmente válida para a transição de um para zero.

Nessa situação o contador CTR2 terá chegado a seu valor máximo (no exemplo = $11_b = 3_d$ e seu sinal de “*carry out*” estará em um, mantendo sua contagem desabilitada (entrada G5 em zero desabilita esse contador).

Suponhamos inicialmente que o sinal de entrada transicione para um e fique estável durante vários períodos do clock CLK. Na próxima transição positiva de CLK, o FF0 mudará sua saída para um. Com isso as entradas da porta XOR ficarão diferentes e sua saída será um. Isso habilitará a carga síncrona do valor zero no contador, que deverá ocorrer na próxima transição positiva de CLK, ou seja, o contador será zerado sincronamente.

Passado um período de CLK, ocorre sua próxima transição positiva e a saída do FF1 transiciona

para um. Essa transição de CLK faz com que o contador seja zerado, como observado anteriormente.

Como agora tanto o FF0 como o FF1 têm o mesmo valor um em suas saidas, a saída da porta XOR estará em zero, permitindo o avanço do contador nos próximos clocks.

Nas próximas três transições positivas de CLK, se o sinal de entrada se mantiver sempre estável, o contador avançará para o seu valor máximo = 11_b = 3_d e, com isso habilitará seu sinal "*carry out*" que inibirá a contagem. Até esse momento então, terão transcorrido 4 períodos do sinal CLK.

O sinal "*carry out*" do contador também habilitará a operação do FF2 (colocando sua entrada WE (Write Enable) em um, de maneira que, transcorrido mais um período de CLK (o quinto já, desde a estabilização da entrada), o FF2 colocará um em sua saída. Então o sinal de entrada só aparecerá na saída se se mantiver estável por, no mínimo, 5 períodos de CLK.

Foram portanto necessários cinco períodos de CLK com o sinal de entrada estável, para que esse sinal aparecesse na saída. Se, porém, ao longo desses cinco períodos o sinal voltasse para zero, o contador seria zerado e a contagem recomeçaria, visto que, nesse caso, a saída da porta XOR iria novamente para um.

Todo o raciocínio anterior é igualmente válido para uma transição do sinal de entrada de um para zero.

No caso de chaves mecânicas usadas em circuitos digitais, o repique dura em torno de 10 a 30 ms., de maneira que um período de 10 ms. para o sinal CLK é razoável.

3. Implementação em VHDL

A Figura 2 a seguir mostra o código fonte em VHDL que implementa o mesmo circuito mostrado anteriormente. Nesse caso foi utilizada a palavra-chave GENERIC para parametrizar a quantidade de bits do contador de maneira acomodar possíveis sinais de relógio disponíveis. No exemplo específico o contador foi definido com dois bits, como no circuito anterior. Nas linhas 28 e 29, a diretiva GENERIC define *counter_size* como valendo 1. Na linha 39 o signal *counter_out* é definido como sendo um vetor com os bits *counter_size downto 0*, ou seja *1 downto 0*, o que significa um vetor de 2 bits. A vantagem da definição de constantes com a diretiva GENERIC é que a definição ocorre em um único local, enquanto que a utilização pode ocorrer em vários lugares. Desejando-se alterar a definição, basta alterar o valor na diretiva GENERIC e todas utilizações serão automaticamente alteradas. Esse processo é semelhante ao uso de diretivas *#define* em linguagem C.

```
23  LIBRARY ieee;
24  USE ieee.std_logic_1164.all;
25  USE ieee.std_logic_unsigned.all;
26
27  ENTITY debounce_v1 IS
28    GENERIC(
29      counter_size  :  INTEGER := 1); -- generaliza capacidade do contador
30    PORT(
31      clk     : IN  STD_LOGIC;  --entrada relógio
32      button  : IN  STD_LOGIC;  --input signal to be debounced
33      result  : OUT STD_LOGIC);
34  END debounce_v1;
35
36  ARCHITECTURE logic OF debounce_v1 IS
37    SIGNAL flipflops   : STD_LOGIC_VECTOR(1 DOWNTO 0); -- flip-flops FF0 e FF1
38    SIGNAL counter_set : STD_LOGIC;                    -- condição xor para reset síncrono
39    SIGNAL counter_out : STD_LOGIC_VECTOR(counter_size DOWNTO 0) := (OTHERS => '0');
40  BEGIN
41
42    counter_set <= flipflops(0) xor flipflops(1);   -- reseta cnt se amostras diferem
43
44    PROCESS(clk)
45    BEGIN
46      IF(clk'EVENT and clk = '1') THEN
47        flipflops(0) <= button;
48        flipflops(1) <= flipflops(0);
49        If(counter_set = '1') THEN                   -- reseta contador se amostras diferem
50          counter_out <= (OTHERS => '0');
51        ELSIF counter_out /= "11" THEN
52          counter_out <= counter_out + 1;
53        ELSE                                         -- contagem estabilizada foi atingida
54          result <= flipflops(1);
55        END IF;
56      END IF;
57    END PROCESS;
58  END logic;
59
```

Figura 2: Código fonte VHDL para debouncing digital.

4. Simução no Quartus

A Figura 3 abaixo mostra o resultado de uma simulação no Quartus, em que a a entrada ficou estável durante vários períodos de tempo, mas somente com 5 períodos de estabilidade o sinal apareceu na saída.

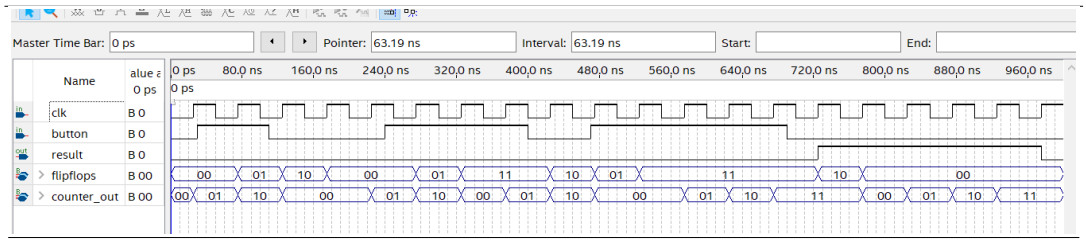

Figura 3: Resultado da simulação no Quartus

Dada a frequência F do sinal de relógio e dado N o número de bits do contador, o intervalo de tempo P em que o sinal de entrada precisará estar estável para aparecer na saída, no pior caso, será:

$$P = \frac{(2^N + 2)}{F}$$

Tanto o arquivo para o LogiSim-evolution com esse circuito implementado também em um bloco VHDL, como o projeto do Quartus estão contidos no arquivo "*Debounce.zip*" que acompanha este tutorial.